



Système de navigation autonome sur TurtleBot 3 Burger via framework ROS

Yue Bai et Yiqiao Gong
03/02/2025

Encadré par Sébastien Bilavarn

Introduction

Ce projet vise à concevoir et à mettre en œuvre un système de navigation en temps réel basé sur le framework ROS, et à construire finalement un système de démonstration complet sur la plateforme TurtleBot 3 Burger. Le projet est divisé en deux principales phases :

Dans la première phase, sous l'environnement Ubuntu 18, la cartographie en 2D est réalisée en utilisant ROS. Après avoir comparé les algorithmes Hector, GMapping et Cartographer, nous avons finalement choisi l'algorithme Cartographer. Les données sont collectées par un LiDAR et transmises à une station de travail distante via le modèle de communication de ROS, permettant ainsi la mise en œuvre d'un SLAM en temps réel et la construction d'une carte de haute précision dans un environnement intérieur.

Dans la deuxième phase, sous Ubuntu 20 et sur la plateforme ROS Noetic, le développement du système de navigation autonome est effectué. Une fois la carte générée, nous avons suivi les étapes du tutoriel officiel du TurtleBot Burger pour configurer progressivement le système, permettant au robot d'éviter les obstacles et de naviguer automatiquement vers des points cibles prédéfinis. Finalement, le système a réussi à démontrer une boucle complète allant de la construction de la carte à la navigation autonome en temps réel.

Pendant la phase de cartographie, le cœur du projet repose sur le choix et l'application de l'algorithme Cartographer. Cela comprend l'acquisition en temps réel des données LiDAR, leur prétraitement et filtrage, ainsi que l'utilisation du mécanisme de publication/abonnement de ROS pour garantir une transmission stable des données. L'actualisation et la visualisation de la carte sont effectuées via des outils tels que RViz.

Dans la phase de navigation autonome, les technologies clés incluent la planification de trajectoire basée sur la carte générée et les stratégies d'évitement d'obstacles. L'environnement a été configuré en suivant strictement les exigences du stack de navigation officiel du TurtleBot Burger, garantissant ainsi que le robot puisse identifier précisément les obstacles dans un environnement intérieur complexe et planifier en temps réel le chemin optimal pour assurer une navigation autonome stable.

Environnement système et choix technologiques

- Description de l'environnement de développement

Phase de cartographie en 2D : **Ubuntu 18 + ROS** (version de ROS et configuration de l'espace de travail).

Phase de navigation autonome : **Ubuntu 20 + ROS Noetic**.

- Plateforme matérielle

Robot TurtleBot Burger utilisé pour la navigation autonome.

- Choix des logiciels et des algorithmes

Comparaison des algorithmes candidats (Hector, GMapping, Cartographer) et choix final de **Cartographer** pour la cartographie.

Configuration de la navigation autonome basée sur le tutoriel officiel du TurtleBot.

Première partie : Création de la carte

Dans la première phase de ce projet, nous utilisons la technologie SLAM pour construire une carte intérieure en 2D, fournissant des données environnementales de haute précision pour la navigation autonome. La cartographie est réalisée avec un LiDAR RP portable, sans assistance d'un odomètre à roues, sous Ubuntu 18 + ROS.

Après avoir comparé HectorSLAM, Gmapping et Cartographer, nous avons choisi Cartographer pour ses avantages :

- **Détection de boucles et optimisation globale** : réduction des erreurs cumulatives et cohérence de la carte sur de grandes surfaces.

- **Cartographie haute précision** : meilleure gestion des environnements similaires comme les longs couloirs.
- **Faible dépendance à l'odométrie** : adapté aux systèmes basés uniquement sur les données LiDAR.

Ubuntu 18 a été retenu pour sa meilleure compatibilité avec Cartographer, garantissant une configuration plus stable. La navigation autonome est ensuite réalisée sous Ubuntu 20 + ROS Noetic. Les deux phases sont interconnectées via des fichiers de carte standardisés (incluant le fichier de configuration YAML et l'image de la carte), assurant un fonctionnement en boucle complète du système.

1) Installation et configuration de Cartographer

Téléchargez et installez les paquets liés à Cartographer, accédez au répertoire racine de l'espace de travail Catkin et compilez avec la commande : **catkin_make**.

Recherche du fichier Launch : Dans l'espace de travail Catkin, accédez au répertoire **launch** du paquet **cartographer_ros**, trouvez le fichier launch utilisé pour démarrer Cartographer, puis modifiez ses paramètres, y compris le port série du LiDAR, le débit en bauds, le nom du topic des données et les paramètres de construction de la carte, comme illustré ci-dessous :

```
C:\Users\gong yiqiao\Desktop> catkin_ws\src> mapping> launch> cartographer_rplidar.launch
1 <launch>
2 <include file = "${find rplidar_ros}/launch/view_rplidar.launch">
3 </include>
4 <arg name = "robot_name" default = "turtlebot" />
5 <arg name = "scan_topic" default = "scan" />
6 <arg name = "odom_topic" default = "${arg robot_name}/odom" />
7 <arg name = "laser_frame_id" default = "laser" />
8 <arg name = "global_frame_id" default = "map" />
9 <arg name = "base_frame_id" default = "${arg robot_name}/base_link" />
10 <arg name = "odom_frame_id" default = "${arg robot_name}/odom" />
11 <node pkg = "tf" type = "static_transform_publisher" name = "base_link_to_laser"
12 | args = "0.027 0 0.0705 0 0 0 ${arg base_frame_id} ${arg laser_frame_id} 100"/>
13 <param name="use_sim_time" value="false"/>
14 <node pkg="cartographer_ros" type="cartographer_node" name="cartographer_node" output="screen"
15 | args="-configuration_directory $(find mapping)/config -configuration_basename rplidar.lua">
16 <remap from="scan" to="${arg scan_topic}"/>
17 </node>
18 <node name="cartographer_occupancy_grid_node" pkg="cartographer_ros"
19 type="cartographer_occupancy_grid_node"/>
20 </launch>
```

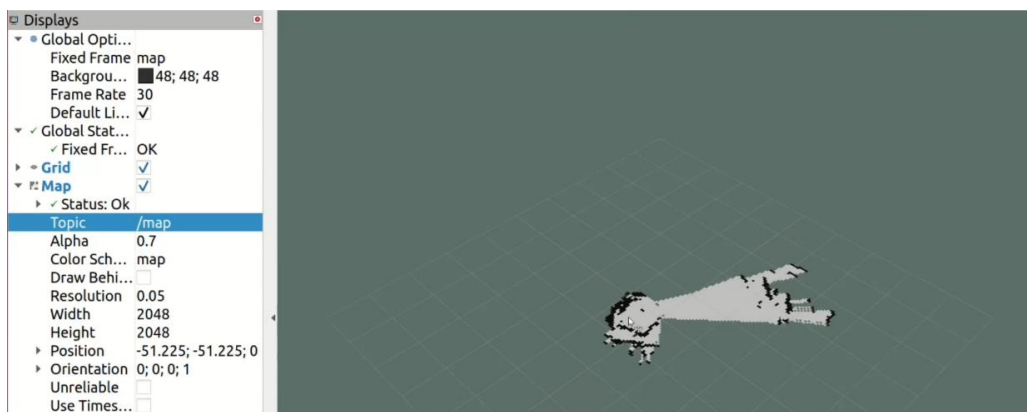
2) Commande de démarrage

Utilisez la commande suivante pour lancer le fichier launch :

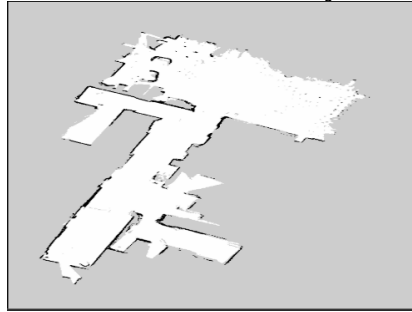
roslaunch cartographer_rplidar.launch

Après le démarrage, le système lancera automatiquement le nœud du pilote LiDAR et le nœud SLAM de Cartographer, et commencera à recevoir les données du LiDAR.

Lancez RViz et ajoutez les topics correspondants (comme /scan, /map, /tf, etc.) dans sa configuration pour s'abonner aux données en temps réel. Cela permet de visualiser directement le flux de données du LiDAR ainsi que la carte en cours de construction, et de vérifier que rplidar est bien démarré et publie les données correctement.



Une fois le système stabilisé, déplacez le LiDAR portable dans l'environnement cible. À ce moment-là, l'algorithme **Cartographer** reçoit en temps réel les données du LiDAR, et construit progressivement la carte 2D grâce à l'appariement des scans et à l'optimisation globale. Pendant ce processus, **RViz** est utilisé pour surveiller la construction en temps réel, en observant l'assemblage de la carte et la détection de boucles. Nous avons cartographié l'étage du **foyer du bâtiment A de Polytech**, et la carte obtenue est la suivante :



Nous avons utilisé l'outil de sauvegarde de carte fourni par ROS pour exporter la carte sous la forme d'un fichier de configuration **YAML** standard et d'une image de la carte. La commande est la suivante :

```
roslaunch map_server map_saver -f my_map
```

Les fichiers de carte enregistrés constituent le résultat final de la **première partie** et seront utilisés dans la **deuxième partie**.

Deuxième partie : Configuration et mise en œuvre de la navigation autonome avec TurtleBot3

Nous avons suivi le **tutoriel officiel de TurtleBot3** pour la configuration : [TurtleBot3](https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/)

(<https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/>)

Sur le PC distant (Remote PC), nous installons l'environnement logiciel nécessaire, comprenant Ubuntu, ROS Noetic et ses dépendances associées.

1. Installation du système et de ROS sur le PC distant

1) Installation du système Ubuntu

Tout d'abord, nous avons téléchargé l'image ISO de **Ubuntu 20.04 LTS Desktop** depuis le site officiel d'Ubuntu(<https://releases.ubuntu.com/20.04/>). Ensuite, nous avons suivi le guide d'installation officiel pour configurer le système, en veillant à ce qu'il démarre correctement et soit connecté à Internet.

2) Installation de ROS Noetic sur le PC distant

Après l'installation et la connexion à Ubuntu, nous ouvrons un terminal et exécutons les commandes suivantes pour mettre à jour et améliorer le système :

```
sudo apt update
```

```
sudo apt upgrade
```

Pour simplifier l'installation de **ROS Noetic**, nous avons trouvé un **script d'installation automatique** sur le GitHub officiel de **ROBOTIS** (https://github.com/ROBOTIS-GIT/robotis_tools) et l'avons téléchargé avec la commande wget :

```
wget https://raw.githubusercontent.com/ROBOTIS-GIT/robotis_tools/master/install_ros_noetic.sh
```

Après le téléchargement, nous rendons le script exécutable :

```
chmod 755 install_ros_noetic.sh
```

Enfin, nous exécutons le script :

```
bash ./install_ros_noetic.sh
```

Ce script installe automatiquement tous les composants nécessaires de **ROS Noetic**, y compris les paquets de base et les dépendances. Une fois l'installation terminée, nous avons exécuté la commande :

```
source ~/.bashrc
```

ou nous avons chargé manuellement le fichier de configuration ROS dans le terminal, afin d'activer les variables d'environnement ROS.

3) Installation des paquets ROS nécessaires pour TurtleBot3

3.1) Installation des dépendances de navigation et de robotique

En nous référant au **Wiki officiel de TurtleBot3**, nous avons exécuté la commande suivante pour installer les paquets essentiels. Cette étape garantit l'installation des fonctionnalités de télécommande, du traitement des données LiDAR et des algorithmes de navigation :

```
sudo apt-get install ros-noetic-joy ros-noetic-teleop-twist-joy \  
ros-noetic-teleop-twist-keyboard ros-noetic-laser-proc \  
ros-noetic-rgbd-launch ros-noetic-depthimage-to-laserscan \  
ros-noetic-rosserial-arduino ros-noetic-rosserial-python \  
ros-noetic-rosserial-client ros-noetic-rosserial-msgs \  
ros-noetic-amcl ros-noetic-map-server ros-noetic-move-base \  
ros-noetic-urdf ros-noetic-xacro ros-noetic-compressed-image-transport \  
ros-noetic-rqt-image-view ros-noetic-gmapping ros-noetic-navigation \  
ros-noetic-interactive-markers
```

3.2) Installation des paquets spécifiques à TurtleBot3

Toujours en suivant le **manuel officiel de TurtleBot3**, nous avons installé **Dynamixel SDK**, les **messages ROS de TurtleBot3** et le **paquet principal TurtleBot3** avec les commandes suivantes :

```
sudo apt install ros-noetic-dynamixel-sdk  
sudo apt install ros-noetic-turtlebot3-msgs  
sudo apt install ros-noetic-turtlebot3
```

Grâce à ces étapes, nous avons réussi à **installer et configurer ROS Noetic** et toutes les **dépendances essentielles pour TurtleBot3** sur le **PC distant (Ubuntu 20.04)**. Ces configurations fournissent une base solide pour la simulation, le contrôle et les expériences de navigation du robot.

2. Configuration du réseau

Pour permettre la communication entre TurtleBot3 et le PC distant, il est nécessaire de configurer correctement la connexion réseau, notamment en obtenant l'adresse IP du PC distant et en définissant les variables d'environnement ROS.

1) Obtention de l'adresse IP du PC distant

Tout d'abord, nous connectons le **PC distant** au **réseau local** via Wi-Fi ou Ethernet et nous nous assurons qu'il peut accéder à Internet. Ensuite, nous ouvrons un terminal et exécutons la commande suivante pour afficher l'adresse IP actuelle de l'appareil :

Ifconfig

Si la connexion se fait via **Wi-Fi**, l'adresse IP peut être trouvée sous l'interface **wlp2s0** ou un nom similaire, dans le champ **inet**.

2) Modification des variables d'environnement ROS

Ouvrir le fichier .bashrc :

nano ~/.bashrc

Ajouter les lignes suivantes à la fin du fichier : (Remplacer par les valeurs correspondant à votre réseau, Si vous souhaitez essayer)

Sauvegarder et recharger le fichier .bashrc :

```
120 source /opt/ros/noetic/setup.bash  
121 export ROS_MASTER_URI=http://172.20.10.2:11311  
122 export ROS_HOSTNAME=172.20.10.2
```

Sauvegarder et recharger le fichier **.bashrc** :

source ~/.bashrc

Grâce à cette configuration réseau, nous avons **assuré que le PC distant et TurtleBot3 sont connectés au même réseau local**, et configuré le **PC distant comme ROS Master**. Ainsi, lors de l'exécution des différents nœuds ou fichiers de lancement de **TurtleBot3**, les autres appareils pourront communiquer avec le **PC distant** en définissant correctement **ROS_MASTER_URI** et **ROS_HOSTNAME**.

Références : Documentation officielle d'**Ubuntu** pour la configuration réseau et **manuel officiel de TurtleBot3** (**Références** : Documentation officielle d'**Ubuntu** pour la configuration réseau et **manuel officiel de TurtleBot3**.<https://ubuntu.com/tutorials>)

3. Configuration de TurtleBot3 Navigation – Côté SBC (Raspberry Pi)

1) Préparation de la carte microSD

Pour configurer le **SBC (Raspberry Pi) de TurtleBot3**, nous avons d'abord préparé une **carte microSD**. Ensuite, nous avons téléchargé depuis la documentation officielle **TurtleBot3** ou le **Robotis E-Manual**, l'image système ROS Noetic compatible avec le **Raspberry Pi 4**.

Une fois le téléchargement terminé, nous avons extrait un fichier **.img** et utilisé **Raspberry Pi Imager** (*installable avec `sudo apt install rpi-imager` ou disponible sur le site officiel*) pour flasher la carte microSD.

2) Configuration du Wi-Fi pour le Raspberry Pi

Après le flashage, nous avons configuré la connexion Wi-Fi du Raspberry Pi en accédant au fichier **netplan** sur la carte SD, généralement situé dans :

/media/\$USER/rootfs/etc/netplan/50-cloud-init.yaml

Nous avons ouvert le fichier avec nano :

nano /media/\$USER/rootfs/etc/netplan/50-cloud-init.yaml

```
24 network:
25   version: 2
26   renderer: networkd
27   ethernets:
28     eth0:
29       dhcp4: yes
30   wifis:
31     wlan0:
32       dhcp4: yes
33       optional: true
34       access-points:
35         yy:
36           password: 12345678
```

Puis, nous avons remplacé **SSID** et **WIFI_PASSWORD** par le nom du réseau Wi-Fi et son mot de passe réels, en respectant strictement la syntaxe **YAML**. Une mauvaise mise en forme entraînerait l'échec de la configuration.

Une fois les modifications enregistrées, au redémarrage du Raspberry Pi, il se connectera automatiquement au Wi-Fi défini, facilitant ainsi la communication réseau avec le PC distant via **ROS**.

3) Configuration du réseau ROS sur le Raspberry Pi

Après l'installation du système et la configuration du Wi-Fi, nous avons défini la configuration réseau ROS pour permettre la communication entre le **Raspberry Pi (SBC)** et le **PC distant**.

Nous avons d'abord obtenu l'adresse IP du **Raspberry Pi** en exécutant :

```
ifconfig
```

L'adresse se trouve sous **wlan0** (Wi-Fi) ou **eth0** (Ethernet).

Ensuite, nous avons ouvert le fichier **.bashrc** et ajouté les variables d'environnement suivantes :

```
export ROS_MASTER_URI=http://172.20.10.2:11311
```

```
export ROS_HOSTNAME=172.20.10.5
```

Puis, nous avons enregistré le fichier et exécuté :

```
source ~/.bashrc
```

Cette configuration permet une communication fluide via **ROS** entre le **PC distant** et le **Raspberry Pi**.

4. Mise à jour du pilote du capteur LDS-02

Le **TurtleBot3** utilisé est équipé du **LiDAR LDS-02**. Pour garantir son bon fonctionnement, nous avons mis à jour son pilote en procédant comme suit :

1) Mise à jour du système et installation de libudev-dev

```
sudo apt update
```

```
sudo apt install libudev-dev
```

2) Téléchargement et mise à jour du driver LDS-02

```
cd ~/catkin_ws/src
```

```
git clone -b noetic https://github.com/ROBOTIS-GIT/ld08\_driver.git
```

```
cd ~/catkin_ws/src/turtlebot3 && git pull cd ~/catkin_ws && catkin_make
```

3) Chargement automatique du capteur LDS-02 au démarrage

```
echo 'export LDS_MODEL=LDS-02' >> ~/.bashrc
```

```
source ~/.bashrc
```

Ainsi, lors du démarrage du **TurtleBot3**, le système reconnaîtra automatiquement le capteur **LiDAR**.

5. Configuration du contrôleur OpenCR pour TurtleBot3 Navigation

L'**OpenCR** est un contrôleur permettant de gérer les moteurs, les capteurs et la communication entre le **SBC (Raspberry Pi)** et le **PC distant**.

1) Connexion et reconnaissance du contrôleur OpenCR

Nous avons connecté l'**OpenCR** au **Raspberry Pi** via un câble **Micro USB** et avons vérifié sa reconnaissance avec la commande :

```
dmesg | grep tty
```

Si le contrôleur est détecté, il apparaît sous **/dev/ttyACM0**.

2) Installation des dépendances et mise à jour du firmware OpenCR

Ajout du support pour **armhf** et installation de **libc6:armhf**

```
sudo dpkg --add-architecture armhf
```

```
sudo apt-get update
```

```
sudo apt-get install libc6:armhf
```

Une fois cela terminé, nous pouvons commencer à télécharger et flasher le firmware d'**OpenCR**. En fonction du modèle de notre robot, nous définissons les variables d'environnement **OPENCNCR_PORT** et **OPENCNCR_MODEL** sur **/dev/ttyACM0** et **burger_noetic** (pour **TurtleBot3 Burger**) :

```
export OPENCNCR_PORT=/dev/ttyACM0
```

```
export OPENCNCR_MODEL=burger_noetic
```

Ensuite, nous supprimons l'ancien fichier compressé du firmware (s'il existe), puis nous téléchargeons la dernière version du firmware depuis le dépôt **OpenCR-Binaries** et nous l'extrayons :

```
rm -rf ./opencr_update.tar.bz2
wget https://github.com/ROBOTIS-GIT/OpenCR-
Binaries/raw/master/turtlebot3/ROS1/latest/opencr_update.tar.bz2
tar -xvf opencr_update.tar.bz2
```

Ensuite, nous entrons dans le dossier `opencr_update` qui vient d'être extrait et exécutons le script `update.sh` afin de transférer le firmware sur la carte OpenCR :

```
cd opencr_update
./update.sh $OPENCRCR_PORT $OPENCRCR_MODEL.opencr
```

Lorsque le message « OpenCR firmware update complete. » apparaît dans le terminal, cela signifie que le firmware a été correctement téléversé sur la carte OpenCR.

Après nous être assurés que la carte OpenCR est correctement connectée et alimentée, nous réalisons un test fonctionnel. Nous plaçons le robot TurtleBot3 sur un sol plat et dégagé, en laissant environ 1 m (soit 40 pouces) d'espace libre. Ensuite, nous maintenons le bouton **PUSH SW1** enfoncé pendant quelques secondes : le robot doit alors avancer d'environ 30 cm (12 pouces). De même, en maintenant le bouton **PUSH SW2** enfoncé, nous observons une rotation d'environ 180° sur place. Si ces deux tests se déroulent normalement, nous pouvons conclure que le contrôleur OpenCR est configuré et fonctionne correctement.

6. TurtleBot3 Navigation – Lancement du bringup

Après avoir terminé l'ensemble des configurations matérielles et logicielles sur le PC distant et sur la SBC (Raspberry Pi), il est nécessaire de démarrer le ROS Master sur le PC distant afin que le TurtleBot3 puisse communiquer avec ce dernier et se préparer aux tâches de navigation ultérieures. Pour ce faire, j'ouvre un terminal sur le PC distant et saisis :

```
roscore
```

Cette commande lance le ROS Master, auquel tous les nœuds ROS se connecteront pour publier et s'abonner aux messages.

Ensuite, nous devons lancer le fichier de bringup de TurtleBot3 sur la SBC (Raspberry Pi). Pour cela, j'ouvre un nouveau terminal sur le PC distant et me connecte en SSH au Raspberry Pi :

```
ssh ubuntu@172.20.10.5
```

Une fois connecté, nous spécifions le modèle de mon TurtleBot3 dans le terminal du Raspberry Pi afin de charger la configuration appropriée :

```
export TURTLEBOT3_MODEL=burger
```

Puis nous lançons la commande suivante, qui active tous les nœuds essentiels du robot : collecte des données capteurs, contrôle du moteur via OpenCR, IMU (unité de mesure inertielle) et publication des données LiDAR :

```
roslaunch turtlebot3_bringup turtlebot3_robot.launch
```

Le terminal affiche alors les paramètres correspondant au modèle choisi (par exemple *burger*), ainsi que des informations relatives à la calibration de l'IMU et à l'initialisation des capteurs, telles que :

```
PARAMETERS
/odom: source
/turtlebot3_core/port: /dev/ttyACM0
/turtlebot3_model: burger
```

Ces messages indiquent que le robot a bien démarré et est prêt à recevoir les instructions du PC distant ou d'autres nœuds ROS.

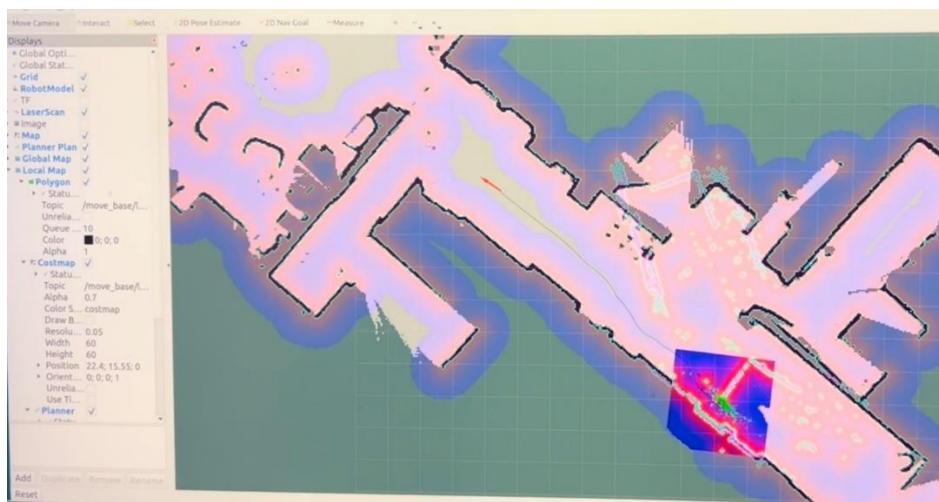
7. Navigation sur TurtleBot3 – Mise en œuvre pratique

Dans les étapes précédentes, nous avons déjà terminé la configuration du système, le paramétrage de la SBC, la configuration d'OpenCR et le lancement du bringup. Dans cette partie, nous allons mettre en œuvre la fonctionnalité de navigation, permettant au TurtleBot3 de se déplacer de façon autonome sur la carte précédemment construite et d'éviter automatiquement les obstacles en cours de route.

Avant de démarrer la navigation, nous avons déjà lancé **roscore** sur le PC distant et le bringup sur la SBC (Raspberry Pi) du TurtleBot3 ; autrement dit, le ROS Master et les divers nœuds de base du robot fonctionnent déjà en arrière-plan.

Sur le PC distant, exécutez la commande suivante :

```
export TURTLEBOT3_MODEL=burger  
roslaunch turtlebot3_navigation turtlebot3_navigation.launch map_file:=$HOME/map.yaml
```



À ce stade, nous pouvons voir que la carte apparaît déjà dans l'interface de RViz. Il nous suffit de définir le point de départ et le point d'arrivée, après quoi le robot se met immédiatement en mouvement. Au cours de son déplacement, on constate qu'il est capable d'éviter automatiquement les obstacles. Pour plus de détails, vous pouvez consulter la vidéo de démonstration.

Conclusion et perspectives

Grâce à ce projet, nous avons construit et intégré un système complet de robot mobile, allant de la création de la carte jusqu'à la navigation autonome. Lors de la première phase de cartographie en 2D, nous avons utilisé Cartographer sous Ubuntu 18 pour scanner l'environnement intérieur et générer une carte de haute précision. Dans la deuxième phase, nous avons déployé cette carte sur la plateforme Ubuntu 20 + ROS Noetic, afin de doter le TurtleBot3 Burger de capacités de planification de trajectoire et d'évitement d'obstacles en temps réel. Tout au long du processus, le framework ROS a fourni un mécanisme efficace pour la collecte de données LiDAR, la construction de la carte, la planification du chemin et la communication à distance. De plus, la navigation officielle de TurtleBot3 ainsi que la configuration du contrôleur OpenCR ont assuré la stabilité du contrôle des moteurs et du traitement des capteurs. Au final, le robot se déplace de manière autonome dans un environnement connu, en atteignant précisément les points cibles prédéfinis.

Bien que le projet ait atteint ses objectifs initiaux, il subsiste de nombreuses pistes d'amélioration et d'extension possibles. Premièrement, il serait judicieux d'ajuster les paramètres des algorithmes de navigation pour

accroître l'efficacité et la robustesse de la planification de trajectoire. Deuxièmement, on pourrait mettre en place un système multi-robots, permettant à plusieurs TurtleBot3 de naviguer simultanément sur une même carte ou de se répartir les tâches, améliorant ainsi la couverture dans des environnements étendus. Troisièmement, on pourrait envisager une mise à jour en temps réel de la carte pendant la navigation, afin de maintenir une représentation de l'environnement toujours à jour et plus précise. Avec la progression continue du matériel et des algorithmes, nous pensons que ce type de système de navigation autonome basé sur ROS trouvera des applications plus larges dans l'industrie, les services et la recherche, offrant ainsi une solution flexible et fiable pour la robotique mobile en intérieur.